Métricas Prometheus — Octopus v1.4.1

Eventos escritos em `/log.txt`, incluindo falhas de PostgreSQL, têm data/hora local e uptime (ou SEM_NTP antes de sincronizar). Isso não altera séries ou contadores Prometheus: cada chamada de log continua sendo um registro.

Netscan periódico e gravação PostgreSQL são configurados separadamente na web. Não alteram a porta ou o IP autorizado do Prometheus. Os erros SQL geram registros de texto no `/log.txt`; não incrementam o contador de falhas de escrita no SD quando o registro foi gravado normalmente. O estado SQL é consultado no painel ou em `netscan_status`, sem novas séries Prometheus nesta versão.

Download e upload do SD ficam no painel da porta 80 (`/sd/download` e `/sd/upload`). Não alteram a porta de métricas nem a autorização do coletor. Usam os contadores HTTP de aplicação existentes; não há nova série por arquivo transferido.

O comando UDP `netscan` executa descoberta em tarefa separada e salva o inventário por MAC em `/netscan.json`; `netscan_status` acompanha a operação. Essa listagem não cria séries Prometheus por equipamento. O exportador mantém a porta configurada e a restrição de coletor existentes durante a varredura.

O formato comum segue o projeto local **ESP32C6**: identidade pelo MAC STA, reset como gauge 1 com código no label, interfaces de IP identificadas, ocupação da partição do firmware e omissão de integrações indisponíveis.

Funcionamento e configuração web

O Prometheus coleta métricas por **pull**, acessando `http://IP_DO_OCTOPUS:PORTA/metrics`. Não há envio para a API do Prometheus, Pushgateway ou remote-write. O painel continua exclusivamente na porta 80.

1. Abra `http://IP_DO_OCTOPUS/` e a aba **Configurações**.
2. Ative **Ativar endpoint de métricas**.
3. Informe o endereço do servidor Prometheus autorizado: IPv4 ou hostname, sem protocolo, caminho ou porta. Em branco, qualquer cliente pode coletar.
4. Escolha a **porta de métricas no Octopus**, padrão 9120. Valores válidos: 1–65535, exceto 80 (painel) e 3232 (OTA). A porta UDP é independente.
5. Salve. A porta é aplicada sem reiniciar e sem alterar as credenciais Wi-Fi.

As Preferences usam o namespace `prometheus`: `enabled` (bool), `host` (String) e `port` (uint16). Os parâmetros são carregados no boot; a coleta inicia desativada em dispositivos ainda não configurados. Salvar parâmetros inválidos retorna HTTP 400 sem alterar a configuração. Falhas de persistência retornam HTTP 500 e são exibidas no formulário.

Quando há endereço de servidor, o IP de origem precisa coincidir com o IP configurado ou resolvido por DNS. Considere o IP observado pelo Octopus se o Prometheus estiver em container, atrás de NAT ou proxy. Hostnames são resolvidos ao iniciar e a cada 60 segundos. Falhas de DNS bloqueiam a coleta com HTTP 503; clientes de outra origem recebem HTTP 403. O erro 403 mostra `IP recebido`, `IP autorizado` e o servidor configurado; a Serial tambem registra a origem bloqueada. Compare esses enderecos antes de alterar a restricao. Abrir o endpoint em outro computador pelo navegador nao testa a permissao do servidor

Prometheus. Para testar a partir do servidor, execute nele `curl http://IP_DO_OCTOPUS:PORTA/metrics`. Em containers/NAT, o endereco observado pode ser diferente do IP esperado. A restrição por IP não é autenticação ou TLS.

O endpoint responde GET com formato Prometheus text 0.0.4, `HELP`, `TYPE`, labels escapados, valores numéricos e quebra de linha final. Não há acesso ao painel ou configuração pela porta de métricas. A página exibe o endpoint, o estado da coleta e um exemplo de configuração com o IP atual do Octopus.

Exemplo no servidor

No `prometheus.yml`, acrescente o job em `scrape_configs` já existente, substituindo IP e porta pelos valores do dispositivo:

```
scrape_configs:
  - job_name: octopus
    scrape_interval: 15s
    scrape_timeout: 5s
    metrics_path: /metrics
    static_configs:
      - targets: ['192.168.0.98:9120']
```

Recarregue a configuração no servidor e confira o target. Para detectar o dispositivo desligado ou inalcançável, use `up{job="octopus"}`, gerado pelo próprio Prometheus. Um dispositivo desligado não consegue exportar `esp_status_online = 0`; essa métrica indica conexão Wi-Fi enquanto responde.

Identificação e disponibilidade

As métricas abaixo são gauges. Informações textuais ficam em labels; não são valores de amostra textuais.

Todas as amostras, incluindo contadores, GPIOs e arquivos do SD, incluem `device="MAC_DO_OCTOPUS"`. O identificador é o MAC station do dispositivo, no mesmo padrão dos demais ESP32. O MAC é lido da interface STA no boot e permanece estável no modo AP. Em `esp_info`, `model="Octopus"` identifica o modelo; `device` contém o MAC, não o nome.

Exemplo ilustrativo:

```
esp_uptime_seconds{device="AA:BB:CC:DD:EE:FF"} 290.369
esp_last_restart_reason{device="AA:BB:CC:DD:EE:FF",reason="6"} 1
esp_ip_address{device="AA:BB:CC:DD:EE:FF",ip="192.168.0.98",interface="sta"} 1
gpio_digital_output{device="AA:BB:CC:DD:EE:FF",pin="15"} 1
sdcard_log_size_bytes{device="AA:BB:CC:DD:EE:FF",file="/udp_log.txt"} 1024
```

No Grafana/PromQL, filtre por `device` para selecionar o equipamento. Após atualizar o firmware, as séries antigas sem esse label deixam de receber amostras; o histórico anterior permanece no Prometheus.

Métrica	Valor e labels
`esp_info`	1; `device` (MAC), `chip`, `model="Octopus"`
`esp_firmware_version`	1; `version`, `build`
`esp_ip_address`	1; `ip`, `interface="sta"` ou `interface="ap"`; uma série por interface ativa
`esp_mac_address`	1; `mac`
`esp_status_online`	1 com station Wi-Fi conectado, 0 em outro estado
`esp_uptime_seconds`	Segundos desde o boot via temporizador de 64 bits
`esp_last_restart_reason`	1; código numérico ESP-IDF no label `reason`
`esp_boot_count`	Boots acumulados em NVS desde a instalação desta versão
`esp_reboot_count`	Boots acumulados menos o primeiro, incluindo ciclos de energia
`esp_watchdog_resets`	Resets TASK_WDT, INT_WDT e WDT acumulados em NVS

Os contadores persistentes são gravados uma vez por boot no namespace `metric_counts`, chaves `boots` e `watchdogs`. Os valores existentes são preservados e saturam em UINT32_MAX. `esp_reboot_count` é derivado de `boots`; a antiga chave `reboots` deixa de ser usada, sem apagar o histórico de boots. Se a leitura ou gravação dos contadores falhar, suas três séries são omitidas e há aviso na Serial. Erase completo de NVS zera esses valores. `reset_wifi` mantém as métricas e a configuração Prometheus.

Rede, CPU e memória

Métrica	Unidade / significado
`esp_wifi_rssi_dbm`	dBm; omitida sem conexão station
`esp_wifi_channel`	Canal da conexão station; omitida sem conexão
`esp_wifi_disconnect_count`	Eventos station-disconnected desde o boot
`esp_wifi_tx_power`	Potência máxima configurada em dBm; conversão de 0,25 dBm
`esp_cpu_frequency_mhz`	MHz
`esp_cpu_usage_percent`	Percentual ocupado médio dos núcleos, amostrado a cada 5 s
`esp_cpu_temperature_celsius`	Temperatura interna do chip; não é ambiente
`esp_task_count`	Quantidade atual de tarefas FreeRTOS
`esp_free_rtos_tasks`	Alias de `esp_task_count`
`esp_heap_free_bytes`	Heap livre
`esp_heap_total_bytes`	Heap total
`esp_heap_minimum_free_bytes`	Menor heap livre desde o boot
`esp_psram_free_bytes`	PSRAM livre; zero sem PSRAM

Métrica	Unidade / significado
`esp_psram_total_bytes`	PSRAM total; zero sem PSRAM
`esp_flash_usage_percent`	Tamanho do sketch / capacidade da partição em execução × 100

Uso de CPU usa os contadores de runtime das tarefas ociosas, sem estimar ocupação pela frequência ou número de tarefas. Sem suporte de runtime stats do FreeRTOS, sem duas amostras completas, com mais de 64 tarefas ou sem snapshot válido, a série é omitida. A leitura usa buffers fixos e deltas por núcleo, incluindo o tratamento de rollover dos contadores. Potência Wi-Fi é omitida sem conexão station ou se o driver falhar; temperatura interna é omitida se a leitura não for finita. Flash é omitida se a partição não for identificada.

Clima de Porto Alegre

Métrica	Labels e significado
`sensor_temperature_celsius`	`source="open_meteo",location="porto_alegre"`; temperatura em °C
`sensor_humidity_percent`	`location="porto_alegre"`; umidade relativa em %
`esp_weather_data_age_seconds`	Segundos desde a última consulta de clima bem-sucedida

Todas também incluem `device` com o MAC. O modelo exportado por `esp_info` é `Octopus`. Use estas consultas no Grafana, com a variável `$device` contendo o modelo selecionado (por exemplo, `Octopus`), não o MAC:

```
sensor_temperature_celsius{source="open_meteo",location="porto_alegre"}
and on(instance, job) esp_info{model="$device"}
```

```
sensor_humidity_percent{location="porto_alegre"}
and on(instance, job) esp_info{model="$device"}
```

```
esp_weather_data_age_seconds
and on(instance, job) esp_info{model="$device"}
```

O `and on(instance, job)` é aplicado na consulta do Prometheus/Grafana, não no texto exportado pelo ESP32. `$device` é substituído pelo Grafana.

A origem é a API Open-Meteo, nas coordenadas aproximadas de Porto Alegre (-30.03, -51.23), campos `temperature_2m` e `relative_humidity_2m` de `current`. Não são sensores locais da placa. A tarefa de baixa prioridade consulta o serviço a cada 15 minutos e repete em 1 minuto após falha, com limites de conexão/leitura e resposta de 16 KiB. O cliente HTTP limitado é baseado no projeto Central, sem bibliotecas JSON ou buffers TLS adicionais; consulta dados públicos por HTTP, como a Central.

O scrape e `age` apenas leem o cache em memória, sem consultar a Internet. Antes do primeiro sucesso, as três séries são omitidas e `age` informa que o clima não foi sincronizado. Falhas não zeram leituras nem a idade; dados com até 30 minutos são mantidos. Depois disso, temperatura/umidade são omitidas, mas a idade continua sendo exportada para evidenciar a defasagem. São rejeitados campos ausentes, null, não numéricos, não finitos ou fora dos limites; com NTP válido, também se rejeita horário da API com mais de uma hora de atraso ou mais de cinco minutos no futuro.

Energia, ambiente local, relés e GPIO

Como no ESP32C6, estas métricas **não são publicadas** enquanto não existir hardware, calibração e driver configurados no firmware:

Métrica	Unidade / significado previsto
`esp_voltage_v`	Tensão de alimentação em V
`esp_current_ma`	Corrente em mA
`esp_power_w`	Potência em W
`esp_battery_percent`	Carga da bateria em %
`esp_battery_voltage_v`	Tensão da bateria em V
`sensor_pressure_hpa`	Pressão atmosférica em hPa
`sensor_air_quality`	Índice do sensor de qualidade do ar
`relay_state`	Estado lógico do relé (0/1)
`relay_activation_count`	Ativações acumuladas
`relay_runtime_seconds`	Tempo ativo acumulado em s
`gpio_analog_value`	Valor bruto de ADC

Métrica digital	Origem da leitura
`gpio_digital_input`	GPIO 0; botão de reset, ativo em LOW
`gpio_digital_output`	GPIOs 2 e 15; LEDs azul e verde

`gpio_digital_input{pin="0"}` exporta o nível lógico do botão (LOW pressionado). `gpio_digital_output` exporta os níveis dos LEDs com labels `pin="2"` e `pin="15"`, além do label comum `device`. Não são alterados ou amostrados pinos SPI/SD como entradas analógicas.

MQTT

Não há cliente MQTT no Octopus atual. `mqtt_connected`, `mqtt_messages_sent`, `mqtt_messages_received` e `mqtt_publish_errors` são omitidas até existir integração real, seguindo o ESP32C6.

Aplicação

Métrica	Tipo / significado
`application_requests_total`	Counter; somente requisições HTTP concluídas desde o boot
`application_errors_total`	Counter; requisições HTTP concluídas com erro
`application_response_time_ms`	Gauge; duração da última requisição HTTP concluída, em ms
`esp_prometheus_scrapes_total`	Counter; coletas autorizadas desde o boot
`esp_udp_commands_total`	Counter; comandos UDP concluídos desde o boot
`esp_udp_listener_up`	Gauge; 1 se a abertura da porta UDP retornou sucesso, 0 se não iniciada ou falhou
`esp_udp_packets_received_total`	Counter; datagramas detectados pelo receptor, antes de leitura e enfileiramento
`esp_udp_replies_sent_total`	Counter; respostas completas aceitas pela pilha UDP local
`esp_udp_send_errors_total`	Counter; tentativas de resposta UDP que falharam
`esp_udp_errors_total`	Counter; erros de comando, falhas TX e descartes por fila cheia
`esp_udp_response_time_ms`	Gauge; duração do último comando UDP concluído, em ms

As métricas `application_*` seguem o escopo HTTP do ESP32C6, incluindo painel, status, configuração web e coleta. UDP tem séries próprias; um comando com várias respostas conta como um comando. Falhas TX são eventos separados, portanto um comando pode gerar mais de um erro. As durações não medem latência de entrega pela rede. A amostra de um scrape ainda não inclui a duração e o contador de requisições concluídas desse próprio scrape. Não são gravados contadores de tráfego em NVS a cada requisição.

`esp_udp_listener_up` reflete o resultado de `_udp.begin()` e a disponibilidade do mutex do socket. Não testa conectividade Wi-Fi nem entrega pela rede. No portal AP, em que o receptor UDP não é iniciado, fica em 0. `esp_udp_packets_received_total` conta cada datagrama detectado por `parsePacket()`, inclusive se depois houver falha de leitura, truncamento ou descarte por fila cheia. Datagramas vazios não reportados pela biblioteca como tamanho positivo não entram nesse contador.

Cada chamada de resposta conta como sucesso ou falha. Sucesso exige `beginPacket()` válido, escrita completa e `endPacket()` aceito. Ausência do mutex, abertura de pacote, escrita parcial ou falha em `endPacket()` incrementam `esp_udp_send_errors_total` e o contador agregado `esp_udp_errors_total`, uma vez por tentativa. O receptor remoto não confirma entrega UDP. Um comando como `sd_read` pode enviar muitas respostas. Esses três novos counters reiniciam no boot, sem persistência em NVS.

Consultas para monitorar a porta e a taxa de tráfego:

```
esp_udp_listener_up and on(instance, job) esp_info{model="$device"}
```

```
rate(esp_udp_packets_received_total[5m])
```

```
rate(esp_udp_replies_sent_total[5m])
```

```
increase(esp_udp_send_errors_total[1h])
```

SD card

Métrica	Tipo / significado
`sdcard_mounted`	Gauge; cartão inicializado no boot
`sdcard_metrics_available`	Gauge; 1 com cache de capacidade/ocupação válido, 0 sem dados válidos
`sdcard_capacity_bytes`	Gauge; capacidade física
`sdcard_total_bytes`	Gauge; tamanho do sistema de arquivos
`sdcard_used_bytes`	Gauge; espaço usado
`sdcard_free_bytes`	Gauge; espaço livre
`sdcard_usage_percent`	Gauge; usado / total × 100
`sdcard_log_size_bytes`	Gauge; label `file="/log.txt"` ou `file="/udp_log.txt"`, somente arquivos legíveis existentes
`sdcard_spi_frequency_hz`	Gauge; frequência SPI configurada, 4000000 Hz no Octopus
`sdcard_append_operations_total`	Counter; appends de log concluídos com sucesso
`sdcard_append_errors_total`	Counter; falhas de append de log, incluindo cartão ausente
`sdcard_log_writes_total`	Counter; registros de log gravados com sucesso desde o boot
`sdcard_log_records_total`	Counter; Log Records, registros gravados com sucesso desde o boot
`sdcard_log_errors_total`	Counter; Log Error Records, registros cuja gravação falhou desde o boot
`sdcard_write_errors_total`	Counter; falhas detectadas de gravação/rotação de logs
`sdcard_log_rotations_total`	Counter; rotações de logs concluídas desde o boot
`sdcard_data_age_seconds`	Gauge; idade do cache válido de capacidade e ocupação
`sdcard_stats_age_seconds`	Gauge; alias de `sdcard_data_age_seconds` mantido para compatibilidade

Sem cartão, montagem e disponibilidade indicam 0; capacidade, ocupação, tamanhos, frequência e idade são omitidas. Um SD montado com estatísticas inválidas mantém `sdcard_metrics_available=0`. Os contadores continuam representando eventos desde o boot. `sdcard_log_writes_total` é equivalente a

`sdcard_append_operations_total`; `sdcard_write_errors_total` também inclui falhas de rotação, enquanto `sdcard_append_errors_total` só conta append.

`sdcard_log_records_total` usa o mesmo contador de `sdcard_log_writes_total`; `sdcard_log_errors_total` usa o contador de `sdcard_append_errors_total`. Incluem chamadas aos logs geral e UDP (um registro pode ter várias linhas). Erros significam falha para gravar um registro: SD ausente, mutex do log não obtido em até 100 ms, abertura ou escrita incompleta, não a presença da palavra ERRO no conteúdo do log. Falhas apenas de rotação continuam em `sdcard_write_errors_total`. Os counters reiniciam no boot, não na rotação ou exclusão dos arquivos; `reset_counters` zera somente os contadores NVS de boot/watchdog.

Consultas para os painéis do exemplo:

```
sdcard_log_records_total and on(instance, job) esp_info{model="$device"}
sdcard_log_errors_total and on(instance, job) esp_info{model="$device"}
```

O cache de espaço é atualizado a cada 30 segundos pela tarefa de processamento, evitando leitura da FAT durante cada scrape. O firmware não detecta e remonta automaticamente um cartão inserido após o boot. Os contadores de escrita cobrem os logs gerenciados pelo StorageManager, não todas as escritas arbitrárias de `sd_write` ou relatórios de crash. Os tamanhos dos logs são atualizados imediatamente nas escritas de log e revisados junto com o cache de espaço.

Migração e watchdog

O schema comum foi alinhado ao `C:/PC/ESP32/ESP32C6/MetricsService.cpp`. Reset passa a ter valor 1 e código numérico no label `reason`; IP recebe `interface`; GPIO usa apenas `device` e `pin`; caminhos dos logs passam a incluir `/`. Séries antigas permanecem no histórico, mas deixam de receber amostras. O percentual de flash passa a representar a partição, e reboot passa a incluir ciclos de energia. Revise consultas que dependiam dos formatos ou significados antigos. A restrição de acesso ao servidor autorizado permanece.

Foi observado TASK_WATCHDOG em WdtRadioCore0, WdtOtaCore0 e IDLE0. O relatório não identifica sozinho a função que bloqueou; é necessário o ELF correspondente para interpretar os endereços do backtrace. Nesta revisão, o tamanho do firmware é verificado uma vez no boot, em vez de reler a imagem durante cada `/status` ou scrape. A consulta de métricas SD copia um snapshot curto em memória, sem esperar pelo mutex durante operações FAT/SD. O laço de rádio bloqueia por pelo menos um tick, garantindo oportunidade de execução às tarefas de menor prioridade mesmo com outra frequência de tick. O watchdog continua ativo. Essas mudanças precisam ser validadas na placa sob coleta e acesso ao painel.

Em 20/10/2026, um novo relatório com SHA `baea6c00f` apontou somente `TaskOtaCore0` sem alimentar o watchdog. O ELF foi localizado em `AppData/Local/arduino/sketches` e o backtrace foi traduzido para `_frxt_dispatch → __yield → readLineWithLimit → WebServer::_parseRequest → WebServer::handleClient → WifiPortalManager::processarPortal → TaskRadioCore0`. O ponto observado era uma espera por dados HTTP, antes do handler da página.

A v1.3.2 usa um override local do hook fraco `yield()`: quando chamado pela tarefa de rádio, bloqueia por um tick para liberar OTA e idle no núcleo 0. Isso corrige a falta de cooperação desse laço com tarefas de menor prioridade, sem alterar o SDK, prioridades, timeout do watchdog ou alimentar o watchdog de outra

tarefa. Não modifica permissões, portas ou métricas. O ELF confirma o ponto da interrupção; ausência de novos resets requer teste de carga na placa.

Na v1.3.3, após o reset `05d15c5be`, o backtrace decodificado mostrou apenas `IDLE1`, sem identificar o ponto onde `WdtRadioCore0` aguardava. Foram tratados conflitos de prazo encontrados no código, sem afirmar a origem exata desse reset:

- `build_opt.h` reduz os limites WebServer de linha para 750 ms e de cabeçalhos para 1500 ms. O Arduino aplica o arquivo também à compilação de Parsing.cpp; uma definição somente no `.ino` não teria esse efeito.
- `RadioWatchdog::Escopo` marca HTTP painel e métricas separadamente e faz checkpoints de progresso antes/depois de cada operação, evitando acumular ambas dentro de uma única janela sem alimentação.
- O hook `yield()` alimenta o watchdog apenas da própria tarefa de rádio, apenas em um escopo HTTP, e somente nos primeiros 2000 ms. A limitação evita alimentação indefinida. O SDK ainda pode aguardar o LF após CR com timeout interno de 5 s; não se promete que toda requisição termine em 1500 ms.
- Aquisição dos mutex UDP e log limitada a 100 ms. Um registro de log perdido por contenção incrementa `sdcard_log_errors_total`, `sdcard_append_errors_total` e `sdcard_write_errors_total`; não é contabilizado como gravação bem-sucedida.
- `crash_info` passa a incluir uma etapa preservada em RTC sem gravações NVS contínuas: HTTP, DNS, leitura/mutex UDP ou log no SD. Operações SD/DNS bloqueadas não recebem alimentação pelo hook HTTP.

Timeout de cinco segundos, panic e watchdogs OTA/processor/idle permanecem ativos. Escrita física no SD e operações síncronas ainda podem exceder esse prazo em falhas reais; o marcador permite diferenciá-las em novo relatório. É necessária validação em hardware, inclusive com clientes HTTP parciais.

Referências

- Modelo de coleta do Prometheus
- Formato de exposição
- Configuração de scrape